

f0697 yang slides (English translation)

f0697 yang slides - Author not stated, Publisher not stated.

- Published: date not stated
- Original: <<https://www.ndss-symposium.org/wp-content/uploads/f0697-yang-slides.pdf>>
- Preserved from: <https://www.ndss-symposium.org/wp-content/uploads/f0697-yang-slides.pdf> (live) on 2026-08-08
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so the page going offline. To read the original, follow the link above.

Content (translated into English)

Machine translation of [f0697-yang-slides.md](#), which holds the source's own words. Code, payloads, type names, URLs and CVE identifiers were masked before translating and restored after, so they are byte-identical to the original.

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Token Time Bomb: Evaluating JWT Implementations for Vulnerability Discovery

Jingcheng Yang¹, Enze Wang¹, Jianjun Chen, Qi Wang, Yuheng Zhang, Haixin Duan, Wei Xie, Baosheng Wang What is a JSON WEB TOKEN (JWT)?

u JSON Web Token — compact, self-contained credential u JSON Web Signature: u Signed for integrity, payload visible u JSON Web Encryption: u Encrypted for confidentiality, payload hidden { { Symmetric "alg": "A256KW", Key "name": "John", "enc": "A256GCM", Public "role": "admin" Content "typ": "JWE", Key } Encryption Key } { { Symmetric "alg": "HS256", "name": "John", Key AES-GCM "typ": "JWS" "role": "admin" Private } } Key AES Key Wrap

Function Base64Encode Base64Encode HMAC Function Base64Encode Base64Encode Base64Encode Base64Encode Base64Encode

JWS Header "." Payload "." Signature JWE Header "." Encrypted_key "." IV "." CiphertText "." Tag

2 Vulnerabilities in JWT implementations

u JWT is widely adopted by industry leaders

...

u JWT is highly flexible and feature-rich, which inadvertently creates opportunities for implementation vulnerabilities

CVE-2023-29357 (Microsoft SharePoint) forged JWT admin privileges remote code execution 3 Our Motivation & Goals

u The majority of JWT vulnerabilities have been discovered manually in prior research u This method may result in some vulnerabilities in JWT implementations being overlooked

How to systematically and efficiently discovering all vulnerabilities in JWT implementations?

4 Research Questions

Research Questions u RQ1: How to generate JWTs to trigger vulnerabilities? u Function-extended Backus-Naur Form (FBNF) for defining JWT grammar u JWT Generator for producing an initial JWT corpus from the FBNF graph u Mutator for expanding input diversity u UCT Update for feedback-driven optimization of generation paths u RQ2: How to detect JWT vulnerabilities automatically? u Differential analysis between impls and within the same impl. u RQ3: What is the prevalence in real-world implementations? u 43 libraries across 10 programming languages u 31 new vulnerabilities, 20 CVEs assigned

5 JWTeemo1: Overview

u Rule Generator u Grammar-based Fuzzing u Parsing FBNF rules and u Utilizing fuzzing with the Mutator and UCT generate FBNF graph Update to enhance the fuzzing efficiency

Rule Generator Grammar-based Fuzzing

Transformer Mutator Harnesses RFCs

Parse Tree

FBNF Graph UCT Update Lexing Parsing FBNF Rules Differencial Syntax Analysis Generator Analyzer FBNF Parser JWT vulnerability detector

1 <https://github.com/JWTeemo/JWTeemo> 6 JWTeemo: Rule Generator

u FBNF Grammar u FUNC: Call functions within grammar rules u IF: Select function based on preceding claim values

u FBNF Parser u Lexing & parsing each FBNF rule into a Concrete Syntax Tree u Transformer merges all CSTs into a unified directed FBNF Graph

7 JWTeemo: Grammar-based Fuzzing

u JWT Generator u Depth-first traversal of the FBNF Graph NodeType Indication

AND Traverse all subtrees

OR Traverse sub-node selected by UCT-Rand algorithm RAND Randomize the number of traversals FUNC Traverse all subtrees and call the function IF Traverse all subtrees and call the function selected by preceding claim values 8 JWTeemo: Grammar-based Fuzzing

u Mutator: Two-level random mutation u Structure: delete or replace a non-terminal node in the FBNF graph u Content: insert or delete a character in a terminal value

u UCT Update: Feedback-driven optimization via UCT-Rand algorithm: u If >50% of implementations accept a JWT, mark the selection as successful u Increase nodes' weight for successful selections u Next traversal use the formula and prefers nodes with higher weight

9 JWTeemo: Grammar-based Fuzzing

u Differential Analyzer u Differences between implementations u Comparing Parsing Results Across Implementations u Differences within the same implementation u Detecting Abnormal CPU

Usage Within an Implementation u Detecting Abnormal Memory Usage Within an Implementation
u Use Chebyshev's inequality to identify statistically significant deviations

10 Evaluation Setup

u Dataset u TIOBE top 16 languages, GitHub stars ≥ 100 , from jwt.io u 43 JWT libraries across 10 programming languages

u Setup u Ubuntu server: 4.1GHz 32-core CPU, 512GB RAM u Harness for each library to receive and verify JWTs u Generated 100,000 JWT test cases

11 Experimental Results: Differences between implementations

u 1,804 differences u 5 types difference u Sign/Encryption Confusion VULN u Algorithm Confusion VULN u JWT Format Confusion VULN u Different Claims Checker SAFE u Different Algorithm Support SAFE

u False Positive: $635 / 1,804 = 35.1\%$

12 Vulnerability 1: Sign/Encryption Confusion

u Sign/Encryption Confusion u Attacker obtains the public key used for JWS verification u Forges a JWE encrypted with this public key, payload set to admin u Vulnerable implementation determines JWT type by counting dots u Uses private key to decrypt attacker's forged payload accepted Web App Login as user Vuln: Bypass Authentication Header: Public Key Private Key {"alg":"RS256"}
Return JWS Payload: if (type == "JWS") Role: user RSA_Verify(JWT, PubKey) Signed with {"role":"user"} Signature: Asymmetric Algorithm Attacker RSA_Sign(PrivKey) Verify JWT With KeyPair if (type == "JWE") Role: admin RSA_Decrypt(JWT, PrivKey)

Header: {"alg":"RSA-OAEP"} JWT Implementation JWE Plaintext: Forge JWE {"role":"admin"}
Encrypted with JWE Ciphertext: RSA_Encrypt(PubKey) Intended Authentication Flow Asymmetric Algorithm Unintended Authentication Flow Using Public key 13 Vulnerability 2: Algorithm Confusion

u Algorithm Confusion u Attacker obtains the public key used for Asymmetric algorithm verification u Attacker changes header alg from RS256 to HS256 u Re-signs the token using the public key as HMAC secret u Vulnerable implementation trusts alg claim, selects HMAC verification u Public key matches as HMAC secret signature accepted Web App Login as user Vuln: Bypass Authentication Header: Public Key Private Key {"alg":"RS256"} Return JWS Payload: if (alg == "RS256") Role: user RSA_Verify(JWT, Key) Signed with {"role":"user"} Signature: Asymmetric Algorithm Attacker RSA_Sign(PrivKey) Verify JWS With Public key if (alg == "HS256") Role: admin HMAC_Verify(JWT, Key)

Header: {"alg":"HS256"} JWT Implementation Payload: Forge JWS {"role":"admin"} Re-Signed with Signature: HMAC_Sign(PubKey) Intended Authentication Flow Symmetric Algorithm Unintended Authentication Flow 14 Using Public key Vulnerability 3: JWT Format Confusion

u JWT Format Confusion u JWT RFC allows only Compact format; JWS RFC also defines JSON format u Some implementations accept JSON-format JWS when processing JWT u Attacker converts JWT to JSON format, inserts forged payload in a custom field u Implementations verifies signature on JSON format JWT passes u App assumes Compact format, extracts claims by dot-splitting reads forged payload Login as user Web App

Vuln: Bypass Authentication

Return JWS header. Verify JWS in Compact Format Status: Valid payload. Verify JWS In Compact Format Verify JWS in JSON Format signature Status: Valid Attacker JWT Implementation

Extract Payload Role: admin { in Compact Format "protected":"header", "payload":"payload", Forge JWS "fake":".b64fakepayload.", Modify JWS to "signature":"signature" Intended Authentication Flow JSON Format } Unintended Authentication Flow 15 Experimental Results: Differences within the same implementation

u Detected 2 types of DoS vulnerabilities: u CPU Exhaustion: Billion Hashes Attack (10 impls) u Memory Exhaustion: Compression DoS (13 impls)

u For example, during fuzzing, JJWT and JWX showed significant resource spikes:

JJWT Library JWX Library 16 Vulnerability 4: Billion Hashes Attack (CPU Exhaustion)

u Billion Hashes Attack u PBES2 algorithm uses p2c claim to specify hash iteration count u Attacker sets p2c to an extremely large value (e.g., 10^9)

u Server performs excessive hash computations during key derivation

Web App Vuln: Denial of Service

CPU Exhaustion Create JWE Using PBES2 alg set p2c = 1000000000 Decrypt JWE Hash iterated p2c times Attacker JWT Implementation

17 Vulnerability 5: Compression DoS (Memory Exhaustion)

u Compression DoS u JWE header zip: DEF indicates payload is compressed u Attacker crafts a JWE with a highly compressed long string as payload u Server decompresses payload after decryption massive memory allocation

Web App Vuln: Denial of Service

Memory Exhaustion Create JWE with a compressed long string Decrypt JWE Decompress long string Attacker JWT Implementation

18 Ablation Study

u Ablation Study: u w/o UCT Update: slower coverage growth, longer time to discover vulnerabilities u w/o Mutator: missed 2 vulnerability types u Both components are essential for effective fuzzing

19 Comparison Study

u Comparison with Existing Tools: u JWT Tool & JWT Editor: lower coverage, only detect known vulnerabilities u JWTeemo: higher coverage, discovers all 5 vulnerability types automatically u Existing tools rely on predefined payloads; JWTeemo discovers unknown vulnerabilities automatically

20 Case Study 1: Authentication Bypass in Kubernetes

u K8s uses JWT-based ServiceAccount tokens for pod authentication u API server extracts iss by dot-splitting (assumes Compact format), but delegates verification to go-jose which also accepts JSON format u Attacker crafts JSON JWT with spoofed fakeiss field; go-jose verifies real signature,

API server reads forged issuer auth bypass u Bug bounty awarded; same vulnerability found in OpenShift Telemeter (CVE-2024-5037) Kubernetes API Server

Extract iss in issuer Compact Format admin

Forge JWS Access Control Mechanism Pod Action

Verify JWS in Status admin Allow Attacker JSON Format Valid JWT LIB Go-JOSE Vuln: Bypass Authentication

21 Case Study 2: Compression DoS in Apache James

u Apache James mail server uses JJWT for SMTP OAuth authentication; u JJWT accepts zip in JWS header (violates RFC) and decompresses payload before signature verification u Attacker sends JWT with compressed bomb, and James decompresses before any verification memory exhaustion, no credentials needed u Vulnerability fixed by Apache

Vuln: Denial of Service

Crafted JWT Remove Decompress Payload Attacker James Mail Signature Part JWT Lib Without Verification Memory Server Exhaustion JJWT 22 Root Cause

u Misunderstanding the Proper Use of JWT Algorithms u Improper use of public/private keys across algorithms u Lack of enforcement on algorithm-key compatibility

u Non-compliant Implementation of JWT Specifications u Supporting JWT formats not allowed by the RFC u Accepting invalid claim usage

u Insufficient Security Warnings for Risky JWT Features u Dangerous claims without explicit limits u Outdated security guidance

23 Mitigation

u For JWT Specification: u Limit p2c claim size to prevent Billion Hashes Attack u Advise against parsing JSON-type JWS in JWT u Suggest upper limit on JWE payload decompression size u Recommend enforcing use claim in JWK to distinguish signing/encryption

u For JWT Implementation Developers: u Strictly bind keys to allowed algorithms and enforce use/alg constraints in JWKs u Avoid supporting excessive or unnecessary features (e.g., JSON format JWS)

u IETF Impact: u Reported mitigations to IETF RFC 8725 authors u Proposals acknowledged and incorporated into draft-ietf-oauth-rfc8725bis-031

1 <https://datatracker.ietf.org/doc/draft-ietf-oauth-rfc8725bis/> 24 Conclusion

u New Framework: JWTeemo u First systematic framework for automated JWT vulnerability discovery

u New Findings: u Evaluated 43 libraries across 10 languages; discovered 31 new vulnerabilities, 20 CVEs assigned

u New Mitigations: u Proposed mitigations adopted by IETF into draft-ietf-oauth-rfc8725bis-03

25 Thank you for listening! Cross-Origin Web Q&A Attacks via HTTP/2 Server Push and Signed HTTP Exchange Jingcheng Pinji Chen Qi Wang Yang

Network and Information Security Lab Tsinghua University yangjc25@mails.tsinghua.edu.cn

